

What is Data Dictionary?

The Data Dictionary is termed as the Logical Data Model for our system. It contains the meta data for all the data that needs to be persisted. The Logical Model uses the concepts defined in a relational theory i.e, DD contains information about all the tables used by the application, the columns in each table and their specifications, the primary-key and foreign-key defined for each table and their specifications. The Logical Data Model information will be stored in an XML file named **data-dictionary.xml**. Defining the data-dictionary is the first step in creating an application using MickeyLite. Follow the guidelines of Data Dictionary during this step.

How to create a data dictionary?

Data Dictionary can be created by creating a file named data-dictionary.xml under a directory (directory name denotes the module name) in <product-home>/conf/ folder. Creating a data dictionary is as simple as writing an XML file. But certain things are to be noted and followed.

Tag Definition

- Comply with the data-dictionary.dtd for mandatory and optional tags.
- Ensure that the "names" associated to "table", "primary-key", "foreign-key" and "column (within a table)" are unique for the overall system.

Naming Conventions

- Only letters, numbers and underscore are allowed in names. Special characters (\$, *, #, %, etc.) are not allowed.
- Avoid using acronyms/abbreviations as much as possible.
- The first character in a name must be a letter.
- Names of the database objects must not contain Spaces.
- Database reserved words should not be used for naming the tables and columns.

Understanding tags

DOCTYPE

It is imperative that you specify the DOCTYPE tag at the beginning of your file to enable DTD validation.

Example : `<!DOCTYPE data-dictionary SYSTEM "data-dictionary.dtd">`

data-dictionary

The `<data-dictionary>` tag is the starting point for the Data Dictionary specification and is used to define the "name" of the system. For example purpose, data-dictionary has been named as Test. It should be same as that of the module-name.

Example : `<data-dictionary name="Test">`

table

The most essential tag for defining a table is the `<table>` tag. Each table tag represents a table structure. A `<table>` tag can contain the following sub tags. table tag can contain an attribute **name** which denotes the table name.

`<column>`

`<primary-key>`

`<unique-keys>`

`<foreign-keys>`

`<indexes>`

Example : `<table name="Employee">`

Guidelines

The name of the tables must be in Title Case (e.g. Employee, StudentDetails)

The table name should be meaningful and convey relevant information (e.g. the 'Employee' table will contain details about the employee)

Underscores should not be used for table names

In case acronyms/abbreviations are used, irrespective of the position of an acronym/abbreviation, its first letter alone must be capitalized and the rest of the letters must be in lower-case. For example,

- MSPAverages --> wrong :: MspAverages --> right
- PollIDToKeyMap --> wrong :: PollIdToKeyMap --> right

Do not append the word 'Table' to any of the table names (e.g. a table must not be called 'StudentDetailsTable' ! Stick to 'StudentDetails')

Limit the table name to a maximum of 20 characters

Table names must be unique

In order to identify that the tables belong to particular application, table names with prefixes can be used. For e.g. SecUser, SecGroup as user table and group table belong to Security application

When a table has been introduced to capture relationships between two entities (either 1:1, 1:n or m:n), this relationship table is to be named as a combination of 2 tables representing the two participating entities. For e.g. in Security application, the two entities 'User' and 'Group' will be represented in 'SecUser' and 'SecGroup' table respectively and the relationship between the two will be represented in 'SecUserGroup' table

column

`<column>` is the tag that is used to define a column. Column definition can be specified by using the below listed tags. To define multiple columns use `<column>` tag within `<columns>` tag. The attribute **name** of the column tag specifies the column name. Persistence supports automatically generating unique values for columns of types BIGINT. If the value of the column has to be generated by persistence, then the same

has to be defined as specified later in `<uniquevalue-generation>` tag.

`<data-type>`

`<max-size>`

`<precision>`

`<default-value>`

`<nullable>`

`<allowed-values>`

Example :

```
<columns>
  <column name="EMP_ID">
    .
  </column>
  <column name="EMP_NAME">
    .
  </column>
</columns>
```

Guidelines

The name of the columns must be in UPPER CASE (e.g. EMPID column in Employee table, STUDID column in StudentDetails table)

Use underscores to separate column names if needed (e.g. DEPARTMENT_ID)

Column names should be as meaningful as possible and convey relevant information

Limit the column names to 20 characters

Columns with data-type BLOB, cannot participate in any constraints (FK, PK, UK and Index) and they also cannot have default-values.

Columns with data-type CHAR with max-size > 255, cannot participate in any Constraints (FK, PK, UK and Index) and they also not have default-value.

Column names must be unique within a table.

If Columns are defined with data-type SCHAR and SBLOB, their values will be stored in the DB in the encrypted form.

For Sequencing columns

- Define Primary Keys first.
- Define Foreign Key columns second.
- Define 'not null' columns third.
- Define 'null' columns fourth.

data-type

The data type for this column. It can be one of the following.

Example : `<data-type>DATE</data-type>`

Data type in data-dictionary.xml

CHAR
INTEGER
TINYINT
BIGINT
BOOLEAN
FLOAT
DOUBLE

Equivalent Java Data Type

java.lang.String
java.lang.Integer
java.lang.Integer
java.lang.Long
java.lang.Boolean
java.lang.Float
java.lang.Double

DECIMAL	java.lang.BigDecimal
DATE	java.sql.Date
TIME	java.sql.Timestamp
DATETIME	java.sql.Timestamp
TIMESTAMP	java.sql.Timestamp
BLOB	java.io.InputStream
SCHAR	java.lang.String
SBLOB	java.io.InputStream

max-size

The maximum length allowed for the data type for this column. The length can be set to -1 if the value size is unknown and varying.

Example : `<max-size>50</max-size>`

precision

The number of digits to be maintained after the decimal point. This is applicable for data-type DECIMAL. If `<precision>` is used, ensure max-size is greater than precision + 2.

Example : `<precision>5</precision>`

default-value

The default value, if any, for this column (`<default-value> ... </default-value>` tag). The following rules have to be followed while specifying default-values:

- You can not specify "null" (or any other String value) as the default value for Numeric columns such as INTEGER, BIGINT, FLOAT and DOUBLE.
- You can not define any default value for BLOB column.

Example : `<default-value>azamsharp@gmail.com</default-value>`

nullable

Can "null" value be specified for this column (this constraint can take the value 'true' or 'false'). **Its generally not advisable to have your primary-key columns as nullable.** So, make sure that the column(s) marked as primary-key for a table have the nullable value set to **false**. By default, table columns can hold null values. Hence, `<nullable>` tag can be avoided, if columns may hold null values.

Example : `<nullable>>false</nullable>`

allowed-values

Can be specific values this column can hold.

Example 1 : By value

```
<allowed-values>
  <value>Male</value>
  <value>Female</value>
</allowed-values>
```

Example 2 : For Range of values

```
<allowed-values>
  <from>A</from>
  <to>C</to>
</allowed-values>
```

Example 3 : By Pattern

```
<allowed-values>
```

```
<pattern>^\w+@[a-zA-Z_]+?\.[a-zA-Z]{2,3}$</pattern>  
</allowed-values>
```

primary-key

Primary key is mandatory for all tables. The primary key constraint is the one which uniquely identifies each record in a database table. A primary key column cannot contain NULL values. There must be only one primary key of 1 - n columns.

Example 1 : Simple Primary Key

```
<primary-key name="Employee_PK">  
  <primary-key-column>EMP_ID</primary-key-column>  
</primary-key>
```

Example 2 : Composite Primary Key

```
<primary-key name="Employee_PK">  
  <primary-key-column>EMP_ID</primary-key-column>  
  <primary-key-column>QUALIFICATION</primary-key-column>  
</primary-key>
```

Guidelines

- Primary Key constraints should be named as TableName_PK for e.g Employee_PK

- A Column participating in PrimaryKey of a table should not contain default-value, allowed-values.

foreign-keys

Foreign keys are the one which are used to relate one table with another. A table can contain 0 - n foreign keys. Each foreign key is defined by using the `<foreign-key>` tag which contains the following attributes.

- name** : name of the foreign key

reference-table-name : the table name that the foreign key is referring to

Example 1 : Simple Foreign Key

```
<foreign-keys>
  <foreign-key name="TeamMembers_FK1" reference-table-name="Team">
    <fk-columns>
      <fk-column>
        <fk-local-column>TEAM_ID</fk-local-column>
        <fk-reference-column>TEAM_ID</fk-reference-column>
      </fk-column>
    </fk-columns>
    <fk-constraints>ON-DELETE-RESTRICT</fk-constraints>
  </foreign-key>
</foreign-keys>
```

Example 2 : Composite Foreign Key

```
<foreign-keys>
  <foreign-key name="TeamMembers_FK2" reference-table-name="Employee">
    <fk-columns>
      <fk-column>
        <fk-local-column>EMP_ID</fk-local-column>
        <fk-reference-column>EMP_ID</fk-reference-column>
      </fk-column>
      <fk-column>
        <fk-local-column>DEPARTMENT_ID</fk-local-column>
        <fk-reference-column>DEPARTMENT_ID</fk-reference-column>
      </fk-column>
    </fk-columns>
    <fk-constraints>ON-DELETE-CASCADE</fk-constraints>
  </foreign-key>
</foreign-keys>
```

Guidelines

Foreign Key constraints should be named as TableName_FK for e.g DEPARTMENT_FK, DEPARTMENT_FK1 etc.

Each FK comprises of FK columns and FK constraints.

Base columns and reference columns defined in FK, must be with same datatype and same max-length.

The foreign key constraints can be one of ON DELETE RESTRICT, ON DELETE CASCADE, ON DELETE SET NULL and ON DELETE SET DEFAULT captured in the `<fk-constraints>...</fk-constraints>` tag. The default is ON-DELETE-RESTRICT. Note: MySQL DB doesn't support ON DELETE SET DEFAULT for INNODB tables.

Always define the slave tables (the one that defines the Foreign Key) after the master table (the one to which the key refers to) . ie., in our earlier example the TeamMembers (the slave) table MUST always be defined after the Employee, Team table (the master table) .

Follow the order of the columns specified in the keys.

- For e.g. If you have defined the Primary Key (TaColumn1,TaColumn2) in a table Table A, in that order then the Foreign Key definition for a table Table B , which references the Table A, should be defined in the same order as Foreign Key(TbColumn1,TbColumn2) References TableA(TaColumn1,TaColumn2) and not in any order like Foreign Key(TbColumn2,TbColumn1) References TableA(TaColumn2,TaColumn1).

For On Delete Cascade

- On delete cascade should be used only for deletion of hierarchical relationships
- Avoid the usage of defining recursive relationships through cascade delete

unique-keys

Unique key means that the combination of such columns are unique. Each unique key has to be defined by `<unique-key>` tag. There can be any number of Unique key columns

defined within a `<unique-key>` definition (more than one `<unique-key-column>...</unique-key-column>` tag). There can be 0 - n unique keys for a table.

Example 1 : Simple Unique Key

```
<unique-keys>
  <unique-key name="Employee_UK1">
    <unique-key-column>EMAILID</unique-key-column>
  </unique-key>
</unique-keys>
```

Example 2 : Composite Unique Key

```
<unique-keys>
  <unique-key name="Employee_UK2">
    <unique-key-column> EMP_NAME</unique-key-column>
    <unique-key-column> DEPARTMENT_ID</unique-key-column>
  </unique-key>
</unique-keys>
```

Guidelines

Unique Key constraints should be named as TableName_UKn, where n is the nth unique-key definition in this table. for e.g Employee_UK1.

Unique Key processing will be skipped for primary key columns.

indexes

`<index>` tag means that the column is indexed. There can be any number of Index columns defined within an index definition. To define multiple indexes, separate index definition has to be used (more than one `<index-column>...</index-column>` tag). The attribute of `<index>` - **name** denotes the index name.

Example 1 : Simple Index

```
<indexes>
  <index name="Employee_IDX1">
    <index-column>DOB</index-column>
  </index>
</indexes>
```

Example 2 : Composite Index

```
<indexes>
  <index name="Employee_IDX2">
    <index-column>EMP_NAME</index-column>
    <index-column>DOB</index-column>
  </index>
</indexes>
```

Guidelines

Indexes should be named as TableName_IDXn, where n is the nth index definition in this table. for e.g Employee_IDX1

You need not index the **key** columns (such as Primary-key, Unique-Key and Foreign-Key). These columns are implicitly indexed. So, the index definition itself is useful only for non-key columns.

Also, the same set of columns should not be indexed more than **once**!

Indexes processing will be skipped for primary key columns.

uniquevalue-generation

Persistence supports automatically generating unique values for columns of types **BIGINT** and **INTEGER**. Persistence will use one of the unique value generator classes available by default, based on the data type specified. Refer [FAQ - General](#), for more

information on this. If the value of the column has to be generated by persistence, then the same has to be defined as shown in example.

Example :

```
<table name="Employee">
  <column name="EMP_ID">
    ....
    <uniquevalue-generation>
      <generator-name>Employee.EMP_ID</generator-name>
    </uniquevalue-generation>
    ....
  </column>
</table>
```

Guidelines

Unique value generators should be named as TableName.ColumnName. For e.g
Employee.EMP_ID

How do I load a data dictionary on server startup (Cold Start)?

After defining DD, a new folder is created for the new module, for example, "Test" in `<server-home>/conf` and the data-dictionary.xml file is placed in `<server-home>/conf/module-name` directory as shown below. The new module must be added to the module.xml file in `<server-home>/conf` folder.

```
Mickeylite or server-home
  o conf
      module-name
      data-dictionary.xml
      module.xml
```

When the Mickeylite server is started, the modules registered with the module.xml file is loaded and the database corresponding to each module is created, as defined in the data-dictionary.xml file.

